Exercícios – Aulas 5 e 6

POO - JAVA: Classes, objetos e encapsulamento

Exercício 1: Sistema de Gestão de Contas Bancárias

Objetivo: Criar uma abstração para uma conta bancária onde o saldo não pode ser alterado diretamente, apenas através de operações de depósito e saque.

1. Classe Modelo (ContaBancaria.java):
   - Atributos privados: numeroConta (int), titular (String) e saldo (double).
   - Construtor: Deve inicializar o número e o titular. O saldo começa em 0.
   - Encapsulamento: Crie apenas o get para o saldo (não deve haver setSaldo, pois o saldo só muda via operações). Crie get/set para o número da conta e o titular.
   - Crie o toString para retornar todos os valores da conta.
   - Métodos: depositar(double valor) e sacar(double valor). O saque só deve ocorrer se houver saldo suficiente.
2. Classe de Execução (BancoApp.java):
   - No método main, instancie uma conta.
   - Tente alterar o saldo diretamente (e observe o erro de compilação).
   - Realize um depósito de R$ 500,00 e um saque de R$ 200,00, exibindo o saldo após cada operação.

Exercício 2: Sistema de Controle de Estoque de Produtos

Objetivo: Controlar produtos em um estoque, garantindo que o preço e a quantidade nunca sejam valores negativos.

1. Classe Modelo (Produto.java):
   - Atributos privados: nome (String), preco (double) e quantidadeEstoque (int).
   - Construtor: Inicializa todos os campos.

- Encapsulamento (Lógica de Validação):
    - No setPreco(double novoPreco), só aceite o valor se for maior que zero.
    - No setQuantidadeEstoque(int novaQtd), só aceite se o valor for maior ou igual a zero.
    - Crie os métodos get correspondentes.
    - Crie o toString para retornar todos os valores do produto.

2. Classe de Execução (EstoqueApp.java):

- Instancie um produto (ex: "Teclado Mecânico", R$ 250.00, 15 unidades).
- Tente definir um preço negativo via setPreco e verifique se a sua lógica impediu a alteração.
- Exiba os dados finais do produto formatados.